

Teste Funcional e Implementação do Programa Identifier

Introdução

Este relatório apresenta a implementação e a validação do programa **Identifier**, atividade prática cujo objetivo é aplicar o critério funcional de **Particionamento em Classes de Equivalência** sobre a lista de casos de teste elaborada na aula anterior, implementar a solução de software correspondente e automatizar a execução dos testes.

Para fins de comparação, a Seção 3 reproduz a lista de casos de teste elaborada na aula anterior. A partir dela, a Seção 4 elabora uma nova lista, seguindo formalmente os passos metodológicos exigidos pela atividade (identificação das classes, derivação dos casos de teste e complementação por valor limite). As seções seguintes descrevem a implementação em **Rust**, os testes automatizados e o resultado real da execução via `cargo test`.

Especificação do Programa

O programa **Identifier** determina se um identificador é válido. Um identificador válido deve:

1. Ter no mínimo 1 caractere e no máximo 6 caracteres de comprimento;
2. Iniciar com uma letra;
3. Conter, nos demais caracteres, apenas letras ou dígitos.

Exemplos de comportamento esperado:

Entrada	Resultado esperado	Justificativa
string	Válido	6 caracteres, começa com letra, demais são letras
stringmuitogrande	Inválido	Mais de 6 caracteres
abc12	Válido	Começa com letra, demais são letras/dígitos
1abc	Inválido	Começa com dígito
ab-c	Inválido	Contém caractere não permitido
(string vazia)	Inválido	0 caracteres

Lista de Casos de Teste da Aula Anterior

Esta seção reproduz, sem alterações, a lista de casos de teste elaborada na aula anterior, mantida aqui apenas para fins de comparação com a nova lista da Seção 4.

Classes de Equivalência (versão anterior)

Variável de Entrada	Classes Válidas	Classes Inválidas
Comprimento (t)	$1 \leq t \leq 6$ (1)	$t < 1$ (2) e $t > 6$ (3)
Inicia com letra (i)	Sim, inicia com letra (4)	Não inicia com letra (5)
Contém letras ou dígitos (c)	Contém apenas letras/dígitos (6)	Contém caractere diferente (7)

Valores de Limite (versão anterior)

- 0 caracteres: abaixo do limite inferior (inválido)
- 1 caractere: limite inferior (válido)
- 6 caracteres: limite superior (válido)
- 7 caracteres: acima do limite superior (inválido)

Conjunto de Casos de Teste (versão anterior)

Entrada	t	i	c	Saída Esperada	Objetivo do Caso
(vazio)	t=0	—	—	Inválido	Limite inferior do comprimento (abaixo do mínimo)
a	t=1	Sim	Sim	Válido	Limite inferior válido do comprimento
abcdef	t=6	Sim	Sim	Válido	Limite superior válido do comprimento
abcdefg	t=7	Sim	Sim	Inválido	Limite superior do comprimento (acima do máximo)
1bcdef	t=6	Não	Sim	Inválido	Isola a classe: não inicia com letra
ab#de	t=5	Sim	Não	Inválido	Isola a classe: contém caractere inválido

Com apenas seis casos, essa versão cobre as sete classes de equivalência identificadas: os quatro primeiros casos exploram a fronteira do comprimento (0, 1, 6 e 7), enquanto os dois últimos isolam, cada um, uma única condição inválida ("não iniciar com letra" e "conter caractere diferente de letra/dígito"), evitando misturar causas de falha em um mesmo teste.

Nova Lista de Casos de Teste

Nesta seção, o critério de Particionamento em Classes de Equivalência é aplicado novamente, de forma mais detalhada, resultando em uma lista mais ampla e diretamente vinculada aos testes automatizados implementados em código.

Passo 1: Identificação das Classes de Equivalência

A especificação permite identificar três variáveis de entrada relevantes: o **comprimento**, o **primeiro caractere** e os **demaís caracteres**. Cada uma delas dá origem a uma classe válida e a uma ou mais classes inválidas.

Comprimento

Classe	Condição	Tipo
C1	0 caracteres	Inválida
C2	De 1 a 6 caracteres	Válida
C3	7 ou mais caracteres	Inválida

Primeiro caractere

Classe	Condição	Tipo
C4	Primeiro caractere é uma letra	Válida
C5	Primeiro caractere não é uma letra	Inválida

Demaís caracteres

Classe	Condição	Tipo
C6	Todos os demais caracteres são letras ou dígitos	Válida
C7	Existe pelo menos um caractere diferente de letra ou dígito	Inválida

Para que um identificador seja válido, as classes C2, C4 e C6 devem ser satisfeitas simultaneamente ($1 \leq \text{comprimento} \leq 6$, primeiro caractere = letra, demais caracteres = letras ou dígitos).

Passo 2: Derivação dos Casos de Teste

Selecionando elementos representativos de cada classe definida no Passo 1, foram derivados os seguintes casos de teste:

ID	Entrada	Comprimento	Primeiro caractere	Demaís caracteres	Resultado esperado	Classes exercitadas
CT01	abc12	5	Letra	Válidos	Válido	C2, C4, C6

ID	Entrada	Comprimento	Primeiro caractere	Demais caracteres	Resultado esperado	Classes exercitadas
CT02	(vazio)	0	—	—	Inválido	C1
CT03	A	1	Letra	—	Válido	C2, C4, C6
CT04	abcdef	6	Letra	Válidos	Válido	C2, C4, C6
CT05	abcdefg	7	Letra	Válidos	Inválido	C3
CT06	1abc	4	Dígito	Válidos	Inválido	C2, C5, C6
CT07	_abc	4	Símbolo	Válidos	Inválido	C2, C5, C6
CT08	ab - c	4	Letra	Contém símbolo	Inválido	C2, C4, C7
CT09	ab@1	4	Letra	Contém símbolo	Inválido	C2, C4, C7

Análise de Valor Limite

A análise de valor limite complementa o particionamento verificando especificamente os valores situados na fronteira da condição de comprimento ($1 \leq t \leq 6$):

Entrada	Tamanho	Relação com o limite	Resultado esperado
(vazio)	0	Abaixo do mínimo	Inválido
A	1	Mínimo permitido	Válido
abcdef	6	Máximo permitido	Válido
abcdefg	7	Acima do máximo	Inválido

Enquanto o particionamento garante que cada classe de comportamento seja exercitada pelo menos uma vez, a análise de valor limite garante especificamente que a implementação trate corretamente as fronteiras 0/1 e 6/7 — pontos em que erros de "off-by-one" costumam ocorrer (por exemplo, usar `<` em vez de `<=`). Os quatro casos de limite (CT02, CT03, CT04, CT05) já estão contidos na lista do Passo 2.

Comparação entre a Lista Anterior e a Nova Lista

Aspecto	Lista da aula anterior (Seção 3)	Nova lista (Seção 4)
Nº de casos	6	9

Aspecto	Lista da aula anterior (Seção 3)	Nova lista (Seção 4)
Classes cobertas	C1–C7 (todas)	C1–C7 (todas)
Estratégia	Combina fronteira de comprimento + isolamento de 2 condições inválidas	Separa explicitamente cada classe, incluindo duas variações de C5 (dígito e símbolo) e duas de C7 (hífen e arroba)
Vínculo com código	Apenas conceitual (papel)	Cada linha corresponde a um teste automatizado em Rust

As duas listas convergem: ambas cobrem as sete classes de equivalência com o mínimo de redundância. A diferença principal é que a nova lista distingue tipos diferentes de caractere inválido no início (dígito vs. símbolo) e no meio da string (hífen vs. arroba), o que aumenta a confiança de que a implementação trata categorias distintas de entrada inválida — não apenas uma delas por classe.

Implementação

A implementação foi realizada em Rust, por meio da função `identifier`, que recebe uma referência para uma string (`&str`) e retorna um valor booleano indicando se o identificador é válido.

```
fn identifier(string: &str) -> bool {
    let mut chars = string.chars();

    let Some(first) = chars.next() else {
        return false;
    };

    if !first.is_ascii_alphabetic() {
        return false;
    }

    if !chars.all(|c| c.is_ascii_alphanumeric()) {
        return false;
    }

    string.len() <= 6
}
```

Funcionamento:

- `string.chars()` cria um iterador sobre os caracteres da entrada.
- `chars.next()` obtém o primeiro caractere; se não existir (string vazia), a função retorna `false` (cobre C1).
- `is_ascii_alphabetic()` verifica se o primeiro caractere é uma letra ASCII (cobre C4/C5).
- `is_ascii_alphanumeric()` verifica se os demais caracteres são letras ou dígitos (cobre C6/C7).
- `string.len() <= 6` verifica o limite máximo de comprimento (cobre C2/C3).

Implementação dos Testes Automatizados

Os testes foram implementados com o framework nativo de testes do Rust, por meio do atributo `#[test]`. A atividade solicita que cada teste deixe explícita a estrutura **Setup / Invocation / Assessment**; no código, essa mesma estrutura é identificada pelos comentários **Arrange / Act / Assert**, nomenclatura equivalente amplamente usada em Rust:

Termo da atividade	Termo usado no código	Papel
Setup	Arrange	Prepara a entrada e o resultado esperado
Invocation	Act	Executa a função <code>identifier</code>
Assessment (Asserts)	Assert	Compara o resultado obtido com o esperado

Exemplo de um dos nove testes:

```
#[test]
fn should_accept_valid_identifier() {
    // Arrange
    let input = "abc12";
    let expected = true;

    // Act
    let result = identifier(input);

    // Assert
    assert_eq!(result, expected);
}
```

Os nove testes automatizados implementados cobrem, um a um, os casos CT01 a CT09 definidos na Seção 4.2.

Relação entre os Casos e as Classes de Equivalência

Classe	Casos que a exercitam
C1 — comprimento 0	CT02
C2 — comprimento de 1 a 6	CT01, CT03, CT04, CT06, CT07, CT08, CT09
C3 — comprimento igual ou superior a 7	CT05
C4 — primeiro caractere é letra	CT01, CT03, CT04, CT08, CT09
C5 — primeiro caractere não é letra	CT06, CT07
C6 — demais caracteres válidos	CT01, CT03, CT04, CT05, CT06, CT07
C7 — presença de caractere inválido	CT08, CT09

Todas as classes identificadas no particionamento possuem pelo menos um caso de teste representativo.

Execução dos Testes

Comando utilizado:

```
cargo test
```

Saída real obtida na execução do ambiente de testes:

```
running 9 tests
test tests::should_accept_identifier_with_one_character ... ok
test tests::should_accept_identifier_with_six_characters ... ok
test tests::should_accept_valid_identifier ... ok
test tests::should_reject_empty_identifier ... ok
test tests::should_reject_identifier_containing_special_character ... ok
test tests::should_reject_identifier_containing_symbol ... ok
test tests::should_reject_identifier_starting_with_digit ... ok
test tests::should_reject_identifier_starting_with_special_character ... ok
test tests::should_reject_identifier_with_seven_characters ... ok

test result: ok. 9 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

Execução do programa principal (`cargo run`), com a entrada de exemplo "string" :

```
string é Válida
```

Resumo: dos 9 casos de teste automatizados (CT01–CT09), **9 foram aprovados e 0 reprovados**, confirmando que a implementação atende à especificação funcional nas sete classes de equivalência identificadas, incluindo os quatro pontos de valor limite (0, 1, 6 e 7 caracteres).

Conclusão

A implementação do programa **Identifier** foi validada por meio do Particionamento em Classes de Equivalência e da Análise de Valor Limite, aplicados sobre a lista de casos de teste da aula anterior e sobre uma nova lista elaborada nesta atividade.

O particionamento permitiu identificar classes relacionadas ao comprimento da entrada, ao primeiro caractere e aos caracteres restantes. A comparação entre as duas listas (Seção 5) mostrou que ambas cobrem as mesmas sete classes de equivalência, ainda que com estratégias de combinação diferentes — a lista anterior prioriza economia de casos, enquanto a nova lista prioriza granularidade e rastreabilidade direta com os testes automatizados.

Os testes automatizados, implementados no framework nativo do Rust com a estrutura Setup/Invocation/Assessment (Arrange/Act/Assert), foram executados via `cargo test`, resultando em **9 aprovados e 0 reprovados**, confirmando que o programa atende à especificação funcional fornecida para o `Identifier`.